

Supplementary materials for the article “Deep learning based prediction of gas chromatographic retention indices for a wide variety of polar and mid-polar liquid stationary phases”: source code of software and parameters of pre-trained models

Dmitriy D. Matyushin, Anastasia Yu. Sholokhova, Aleksey K. Buryak

A.N. Frumkin Institute of Physical Chemistry and Electrochemistry, Russian Academy of Science

e-mail: dm.matiushin@mail.ru

Prediction of gas chromatographic retention indices for various stationary phases

For retention indices on standard non-polar, semi-standard non-polar, and standard polar stationary phases, two deep learning models are used: CNN (convolutional neural network) and MLP (multilayer perceptron). For the best results, the results of these models are averaged. We provide two sets of parameters for these two neural networks: trained for non-polar and trained for polar stationary phases. Pre-trained parameters are provided, while training of these models from a graphical user interface (GUI) is not possible.

For DB-624 and OV-17 stationary phases, a second-level model is used. At the first stage, CNN and MLP predict retention indices for all available standard non-polar, semi-standard non-polar, and standard polar stationary phases. Then these retention indices are used (together with molecular descriptors) as inputs for the second-level machine learning model that predicts retention indices for DB-624 and OV-17 stationary phases. Such models can be trained from GUI.

For DB-1701 and DB-210 stationary phases, a “third-level” model is used. Retention indices for these stationary phases are computed as a linear combination of retention indices for squalane, DB-1, DB-5, DB-624, and DB-Wax stationary phases.

Compiling instructions

This software consists of a Java-based deep learning command line program for retention index prediction and a GUI that is build with the Free Pascal and the Lazarus integrated development environment. The Java-based program is based on our previous software for retention index prediction [1], and the compiling procedure is very similar to that for the previous software.

1) Install prerequisites. You should download and install the following software:

- Java Development Kit, version 11 or above. However, with versions newer than version 16, you can encounter some problems because some of dependencies will not work correctly with newer versions.

In this case, update the versions of dependencies in the `pom.xml` file.

<https://jdk.java.net/>

<https://www.oracle.com/java/technologies/javase-downloads.html>

- Apache Maven (<https://maven.apache.org>), version 3.8 or above.

- Lazarus integrated development environment (<https://www.lazarus-ide.org/>), version 2.0 or above, a corresponding version of Free Pascal Compiler 3.2 or above.

The first two dependencies you can install on Debian or Ubuntu Linux with a single command:

```
sudo apt-get install default-jdk maven
```

However, **DO NOT INSTALL LAZARUS FROM YOUR LINUX REPOSITORY**. Download and install the corresponding packages from the Lazarus website (www.lazarus-ide.org) and follow the instructions on that site. The repositories of Ubuntu and Debian Linux contain very old misconfigured packages of Lazarus. Installation of these packages will not work correctly (as for 2021) and will prevent your system from the correct installation of correct Lazarus packages. For Windows, also download the installer from the Lazarus website.

For Windows, you (optionally) may also need the Microsoft Visual Studio redistributable library.

2) Ensure that all prerequisites are installed properly. Type

```
javac -version
```

```
lazbuild -version
```

```
mvn -version
```

in the command line. On Windows, open a command line by typing `cmd` in the Search field in the Start menu. All three commands should give meaningful output (versions of installed packages). If they do not – most probably you should add paths to binaries to the `PATH` system variable. Instructions for editing the `PATH` variable are available online for your operating system. For Windows, the `PATH` variable can be edited the following way. Go `Control Panel > System and Security > System > Advanced system settings > Advanced tab > Environment Variables button > System Variables block`. Then add to the `PATH` variable the full paths to your JDK binaries (e.g., "`C:\Program Files\Java\jdk-11\bin`", "`C:\Lazarus`", "`C:\Maven\apache-maven-3.6.3\bin`"). Specify the paths where the binaries (e.g., `.exe` files on Windows) are actually located.

3) Download and unpack the required files.

- Download and unpack the sources.zip file from this figshare repository.
- Download and unpack to the same directory the pretrainedModelParametersAndSourceCode.zip file from our previous figshare repository:

<https://www.doi.org/10.6084/m9.figshare.12651680>

<https://ndownloader.figshare.com/files/23851544>

Unpack them in the same directory (the models folder and the App2.java file should be in the same directory).

- Download to the same directory files with pre-trained models: mlp.txt, cnn.txt, mlpPolar.txt, cnnPolar.txt, db624.svr, db17.svr.

4) Compile the software by running compile.bat (on Windows) or compile.sh (on Linux). After a while, the software will be compiled. Now you can run the easy-to-use GUI software from this folder. The executable file will be named retentionindexprediction.exe (on Windows) or retentionindexprediction (on Linux). The software should have permissions to create and delete files in this folder.

You can also compile it manually (recommended if automatic batch scripts do not work for some reason). Copy App2.java, TrainPolar.java, and SecondLevelModelForPolar.java files to the ./src/main/java/ru/ac/phyche/gcmsburyak/retentionprediction4 folder. Create the models_polar folder and copy there db624.svr and db17.svr files. Copy the descriptors_info.txt file from the models folder to the models_polar folder. Then compile the Java command line program with commands

```
mvn clean compile test
```

```
mvn package
```

After that, move the retentionprediction4-0.0.6-jar-with-dependencies.jar file from the target subfolder to the parent folder where the models are unpacked. Convert neural networks parameters from the ASCII format to the binary format using the compiled jar file. Run commands

```
java -Xmx1500M -cp retentionprediction4-0.0.6-jar-with-dependencies.jar ru.ac.phyche.gcmsburyak.retentionprediction4.TrainPolar txt2nn  
./mlp.txt ./cnn.txt ./models_polar/mlp.nn ./models_polar/cnn.nn  
java -Xmx1500M -cp retentionprediction4-0.0.6-jar-with-dependencies.jar ru.ac.phyche.gcmsburyak.retentionprediction4.TrainPolar txt2nn  
./mlpPolar.txt ./cnnPolar.txt ./models_polar/mlpPolar.nn ./models_polar/cnnPolar.nn
```

Then open retentionindexprediction.lpi with Lazarus as a project and run it. It should work.

5) (Optional step, only for Windows) If you want to run the XGBoost model from our previous work [1] on Windows, you should do some additional actions. Read sections 8-11 from the corresponding instruction:

<https://ndownloader.figshare.com/files/25911693>

<https://www.doi.org/10.6084/m9.figshare.12651680>

The use of the command line software

Most of the functions are available from our GUI and it is recommended to use GUI. GUI is self-explanatory. However, it is possible to use the software from the command line and it can be usable in some cases. Run the software with the following command

```
java -cp retentionprediction4-0.0.6-jar-with-dependencies.jar ru.ac.phyche.gcmsburyak.retentionprediction4.App2
```

Parameters are described below.

Usage:

1) Base prediction for 5 common stationary phases (use the default location of the models):

```
.... Predict <a SMILES string or a file with SMILES strings, one per line>
```

When this option is used, models and descriptors info should be located in the ./models_polar folder.

2) Base prediction for 5 common stationary phases:

```
.... Predict5 <folder with neural networks: for polar and non-polar phases, CNN and MLP> <descriptors info file> <second-level model file (SVR for DB624)> <a SMILES string or a file with SMILES strings, one per line>
```

3) To predict retention index using CNN and MLP:

```
.... PredictNN <MLP neural network text file> <CNN neural network text file> <descriptors info file> <stationary phase number (Integer)> <a SMILES string or a file with SMILES strings, one per line>
```

4) To predict retention index using CNN, MLP, and a second-level model:

```
.... PredictSL <folder with four neural networks: for polar and non-polar phases, CNN and MLP> <descriptors info file> <second-level model file> <second-level model type> <a SMILES string or a file with SMILES strings, one per line>
```

5) To make cross-validation for the second-level model:

```
.... CV <folder with four neural networks: for polar and non-polar phases, CNN and MLP> <descriptors info file> <second-level model file> <second-level model type> <data set file> <log file (cross-validation)>
```

6) To train a second-level model for a data set:

```
.... Train <folder with four neural networks: for polar and non-polar phases, CNN and MLP> <descriptors info file> <second-level model file> <second-level model type> <data set file>
```

7) To validate a second-level model for a data set:

```
.... Validate <folder with four neural networks: for polar and non-polar phases,
CNN and MLP> <descriptors info file> <second-level model file> <second-level model
type> <data set file> <predictions file>
```

8) To remove from the data set the compounds that are contained in the other data set:

```
.... RemoveOverlap <data set 1 file> <data set 2 file> <out file (data set 1 after
removing)>
```

The folder with neural networks should contain following files: mlp.nn, cnn.nn, mlpPolar.nn, cnnPolar.nn.

Second-level models: 0 – SVR (support vector regression), 1 – SVR linear, 2 – random forest, 3 – gradient boosting.

Numbers of stationary phases should be specified in accordance with the following tables. For standard and semi-standard non-polar stationary phases:

N	Name	N	Name
0	DB-1	18	HP-5MS
1	SE-30	19	VF-5MS
2	OV-101	20	Squalane
3	OV-1	21	HP-5_MS
4	Methyl_Silicone	22	SE-54
5	CP_Sil_5_CB	23	DB-5MS
6	BP-1	24	Apiezon_L
7	HP-1	25	BPX-5
8	SPB-1	26	SE-52
9	RTX-1	27	CP_Sil_8_CB
10	Ultra-1	28	SPB-5
11	Petrocol_DH	29	RTX-5
12	Polydimethyl_siloxane	30	TR5-MS
13	OV-1,_SE-30,_Methyl_silicone,_SP-2100,_OV-101,_DB-1,_etc.	31	Ultra-2
14	Other_non_polar	32	DB-5_MS
15	5_%_Phenyl_methyl_siloxane	33	ZB-5
16	DB-5	34	SLB-5_MS
17	HP-5	35	Other_semi_standard_non_polar

For standard polar stationary phases:

N	Name	N	Name	N	Name	N	Name
15	DB-Wax	20	PEG-20M	25	Innowax	30	Carbowax
16	Carbowax_20M	21	BP-20	26	Innowax_FSC	31	ZB-Wax
17	Supelcowax-10	22	FFAP	27	RTX-Wax	32	HP-Wax
18	OV-351	23	CP-Wax_52CB	28	PEG_4000	33	AT-Wax
19	HP-Innowax	24	HP-Innowax_FSC	29	DB-FFAP	34	Stabilwax
35	Other_polar						

Most of these stationary phases are very similar (or identical) to each other. However, we give names and types according to specified in the NIST database.

If you want to train MLP and CNN using your data set, use

`java -cp retentionprediction4-0.0.6-jar-with-dependencies.jar ru.ac.phyche.gcmsburyak.retentionprediction4.TrainPolar`

command. It will show the basic usage information. However, you should use the CUDA library. To enable CUDA, you should edit the `pom.xml` file and recompile the project. Check the `Deeplearning4j` instruction for more information.

All options of our previous software [1] are also available in this updated version. Read carefully our previous documentation:

<https://ndownloader.figshare.com/files/25911690>

Information about building using CUDA, about further command-line options and Java main classes, about Java API is given there.

Reference:

(1) Matyushin, D. D.; Buryak, A. K. Gas Chromatographic Retention Index Prediction Using Multimodal Machine Learning. *IEEE Access* 2020, 8, 223140–223155.

<https://doi.org/10.1109/ACCESS.2020.3045047>

<https://www.doi.org/10.6084/m9.figshare.12651680>